Day 85 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 2 reference

Watch: What are Word Embeddings? (IBM Technology) | Word Embedding and Word2Vec, Clearly Explained (StatQuest)

`day85_word_embeddings.py`, segment by segment, in the order `main()` uses things. Day 62's data code is copied verbatim and explained there. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: V = 2,082 vocabulary entries · d = numbers per word · E = the table (V × d) · win = a word id and its two neighbours' ids · H = 64 hidden units

Segment 0 — Setup

Settings

CODE

```
SEEDS = [0, 1, 2]
EPOCHS = 30
LRS = (0.25, 0.5, 1.0, 2.0)
DIMS = (2, 8, 32, 128)
```

- `LRS = (0.25, 0.5, 1.0, 2.0)` — the learning rates tried on validation, for each kind of input separately.
- `DIMS` — the table widths compared, from 2 to 128 numbers per word.

Segment 1 — The window tagger

windows: a word and its neighbours

CODE

```
def windows(ids, mask):
    left = np.concatenate([np.full((len(ids), 1), PAD), ids[:, :-1]], axis=1)
    right = np.concatenate([ids[:, 1:], np.full((len(ids), 1), PAD)], axis=1)
    return np.stack([left[mask], ids[mask], right[mask]], axis=1)
```

- `np.concatenate([np.full(..., PAD), ids[:, :-1]], axis=1)` — shift right by one, PAD in front: each position's left neighbour.
- `left[mask]` — boolean indexing keeps only real words, flattening every sentence into one list.
- `np.stack([...], axis=1)` — three (n,) arrays become one (n, 3) array.

WindowTagger: one-hot or dense inputs

MATH

one-hot input: $[1(w_{-1}); 1(w_0); 1(w_{+1})]$ (3V numbers); dense input: $[E[w_{-1}]; E[w_0]; E[w_{+1}]]$ (3d numbers)

CODE

```
class WindowTagger:
    def __init__(self, V, n_tags, kind="dense", d=32, H=64, lr=0.5, seed=0):
        rng = np.random.RandomState(seed)
        self.V, self.kind, self.lr = V, kind, lr
        width = 3 * V if kind == "onehot" else 3 * d
        if kind != "onehot":
            self.E = rng.randn(V, d) * 0.1
        self.W1 = rng.randn(width, H) / np.sqrt(3 * (d if kind != "onehot" else 1)); self.b1 =
            np.zeros(H)
        self.W2 = rng.randn(H, n_tags) / np.sqrt(H); self.b2 = np.zeros(n_tags)
    def inputs(self, win):
        if self.kind == "onehot":
            X = np.zeros((len(win), 3 * self.V))
            X[np.arange(len(win))[:, None], win + np.arange(3) * self.V] = 1.0
            return X
        return self.E[win].reshape(len(win), -1)
    def forward(self, win):
        self.win, self.X = win, self.inputs(win)
        self.Hh = np.tanh(self.X @ self.W1 + self.b1)
        z = self.Hh @ self.W2 + self.b2
        e = np.exp(z - z.max(axis=1, keepdims=True))
        self.p = e / e.sum(axis=1, keepdims=True)
        return self.p
```

- `width = 3 * V if kind == "onehot" else 3 * d` — a conditional expression decides the input layer's size.
- `X[np.arange(len(win))[:, None], win + np.arange(3) * self.V] = 1.0` — broadcast indexing: one 1 in each of the three V-wide blocks of every row.
- `self.E[win].reshape(len(win), -1)` — (n, 3) ids give (n, 3, d) rows, reshaped to (n, 3d): the three vectors side by side.

step: the backward pass and the table's gradient

MATH

$dE[t] = \Sigma_{\text{positions holding } t}\, dX$; one-hot: $dW_1 = X^T\, dpre$ (only the rows of present words are non-zero)

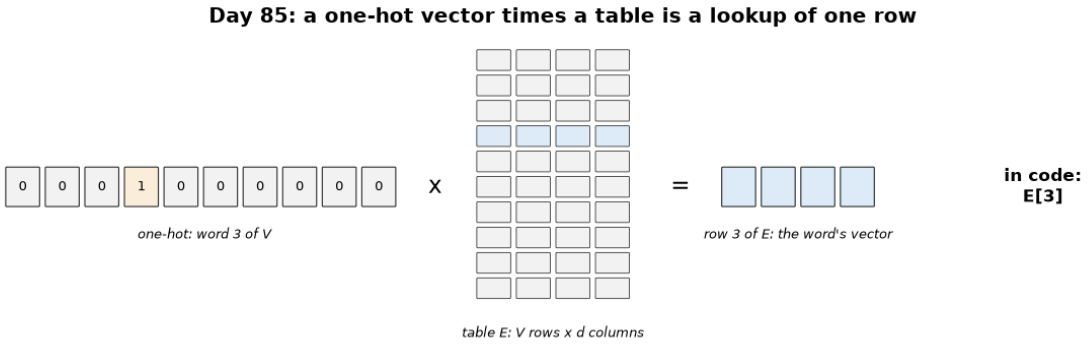
CODE

```
def step(self, win, y):
    p = self.forward(win)
    n = len(y)
    dz = p.copy(); dz[np.arange(n), y] -= 1; dz /= n
    dW2, db2 = self.Hh.T @ dz, dz.sum(axis=0)
    dpre = (dz @ self.W2.T) * (1 - self.Hh ** 2)
    dW1, db1 = self.X.T @ dpre, dpre.sum(axis=0)
    if self.kind == "dense":
        dX = (dpre @ self.W1.T).reshape(n, 3, -1)
        dE = np.zeros_like(self.E); np.add.at(dE, win, dX)
        self.E -= self.lr * dE
    for W, g in ((self.W1, dW1), (self.b1, db1), (self.W2, dW2), (self.b2, db2)):
        W -= self.lr * g
```

- `dz[np.arange(n), y] -= 1` — softmax minus one-hot, in place.
- `np.add.at(dE, win, dX)` — Day 61's scatter-add: a word in two places of a window, or in many windows, collects every gradient.

- `for W, g in ((self.W1, dW1), ...): W -= ...` — tuples of (array, gradient); -= changes each array in place.

SYSTEM DIAGRAM



A one-hot vector times a table is a lookup.

Segment 2 — Experiments

fit: the best epoch on validation

CODE

```
def fit(data, V, n_tags, seed, epochs=EPOCHS, **kw):
    (win_tr, y_tr), (win_va, y_va), (win_te, y_te) = data
    net = WindowTagger(V, n_tags, seed=seed, **kw)
    rng = np.random.RandomState(seed)
    best = (-1.0, None, -1)
    seconds = []
    for epoch in range(epochs):
        start = time.perf_counter()
        perm = rng.permutation(len(y_tr))
        for s in range(0, len(perm), 64):
            idx = perm[s:s + 64]
            net.step(win_tr[idx], y_tr[idx])
        seconds.append(time.perf_counter() - start)
        val = (net.predict(win_va) == y_va).mean()
        if val > best[0]:
            best = (val, copy.deepcopy(net), epoch)
    net = best[1]
    return dict(net=net, val=best[0], epoch=best[2], test=(net.predict(win_te) == y_te).mean(),
                seconds=np.median(seconds))
```

- `copy.deepcopy(net)` — a full copy of the model at its best validation epoch.
- `np.median(seconds)` — the middle epoch time: one slow epoch does not distort it.

demo_lookup: the same rows two ways

CODE

```
one_hot = np.zeros((1000, V)); one_hot[np.arange(1000), ids] = 1.0
print(f"\nlargest |one_hot @ E - E[ids]| over 1,000 words: {np.abs(one_hot @ E -
    E[ids]).max():.1e}")
t_mm = min(timeit(lambda: one_hot @ E) for _ in range(5)); t_lk = min(timeit(lambda: E[ids]) for _
    in range(5))
```

- `one_hot[np.arange(1000), ids] = 1.0` — row i gets its 1 in column ids[i].
- `min(timeit(...) for _ in range(5))` — the fastest of five timings, so a busy moment does not count.

demo_training: a learning rate per kind of input

CODE

```
for kind in ("onehot", "frozen", "dense"):
    vals = [fit_cached(data, V, nt, 0, lr=lr, kind=kind, d=32)["val"] for lr in LRS]
    grid[kind] = LRS[int(np.argmax(vals))]
```

- `[fit_cached(...)["val"] for lr in LRS]` — one run per rate; only the validation accuracy is kept.
- `LRS[int(np.argmax(vals))]` — the rate with the best validation accuracy.

same_tag_share: nearest neighbours by cosine

MATH

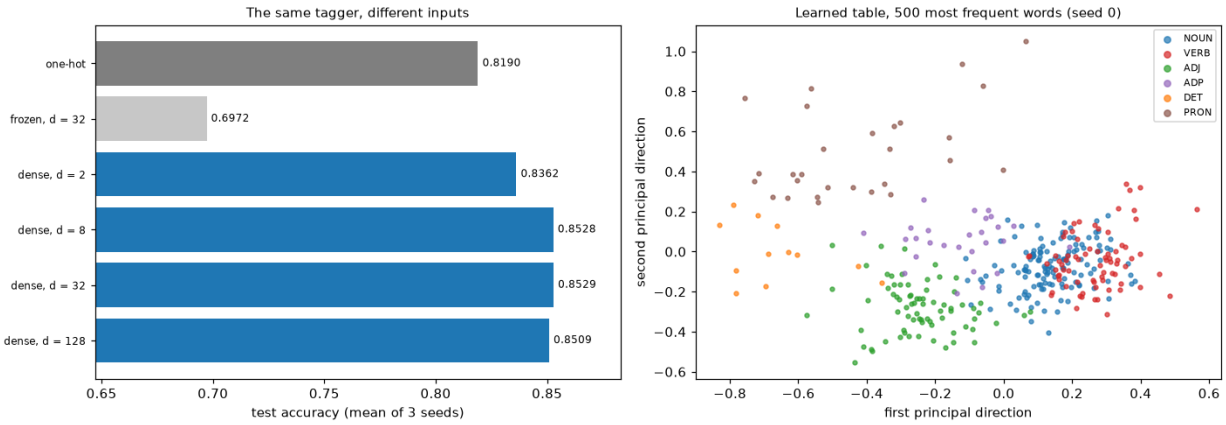
$$cos(u, v) = u \cdot v / (|u| \, |v|)$$

CODE

```
def same_tag_share(T):
    N = T[top] / (np.linalg.norm(T[top], axis=1, keepdims=True) + 1e-12)
    sim = N @ N.T
    np.fill_diagonal(sim, -np.inf)
    return np.mean(tags[top][sim.argmax(axis=1)] == tags[top])
```

- `T[top] / np.linalg.norm(T[top], axis=1, keepdims=True)` — every row scaled to length 1, so dot products are cosines.
- `np.fill_diagonal(sim, -np.inf)` — a word's similarity with itself is excluded.
- `sim.argmax(axis=1)` — each word's most similar other word.

REAL GRAPH



Real graph from the script: accuracy by input, and the learned table in two dimensions.

Quick Syntax Glossary — new or reinforced constructs

- `E[ids]` — an embedding lookup by fancy indexing.
- `np.add.at(dE, ids, dX)` — the lookup's backward pass.
- `a[mask]` — boolean indexing: the selected entries, flattened.
- `np.stack(list, axis=1)` — join arrays along a new axis.
- `np.fill_diagonal(A, v)` — set the diagonal in place.
- `np.linalg.norm(A, axis=1, keepdims=True)` — each row's length, kept as a column.

- `copy.deepcopy(obj)` — an independent copy of a whole object.

End of Day 85 syntax reference. For the experiments and what they showed, see the Day 85 complete guide.